
Rapport de TP

Développement Web API REST & Client Web

Exercice 1 : Client Pokémon / Exercice 2 : Guerre des Animaux

Mouhammad NDIAYE

Ibrahima TOURE

Université Clermont Auvergne

IUT d'Aurillac

Cours : R4-09-VCOD Programmation Web

Professeure : Morgane VIDAL

Année universitaire 2025–2026

Table des matières

1	Introduction	2
2	Exercice 1 Client Web Pokémon	2
2.1	Architecture	2
2.2	Fonctionnalités	2
2.3	Endpoints de la PokéAPI utilisés	3
3	Exercice 2 : Guerre des Animaux	3
3.1	Architecture	3
3.2	Modèles de données	4
3.3	Scraping : Données issues d'ACT'ani	4
3.4	Authentification	4
3.5	Liste des endpoints	5
4	Guide d'utilisation	6
4.1	Exercice 1 Client Pokémon	6
4.2	Exercice 2 Guerre des Animaux	6
	Conclusion	8

Introduction

Ce rapport présente les deux applications développées dans le cadre du TP de la ressource R4-09-VCOD Programmation Web.

1. **Exercice 1** : un client web JavaScript interagissant avec la PokéAPI publique, permettant de rechercher des Pokémon, de les comparer et de consulter les baies.
2. **Exercice 2** : une API REST Django et son client web pour la *Guerre des Animaux*, adaptation de la Guerre des Chats demandée dans le sujet.

Pour l'exercice 2, nous avons choisi d'adapter le thème des chats en y intégrant des animaux au sens large, afin de mettre en pratique les compétences développées en **SAE 4-02-VCOD Collecte automatisée de données web**. Nous avons effectué le scraping du site **ACT'ani** : éducatrice canine et instructrice Mantrailing basée dans le Cantal ([voir ici]) pour extraire les animaux réels de Morgane (Shana, Chopin, Toundra, Echo, Tostaky, Rousseau, Kovou) et les intégrer directement comme données initiales dans la base de données. Cette approche illustre concrètement l'intérêt du web scraping dans un projet applicatif complet.

Exercice 1 Client Web Pokémon

Architecture

L'application est un client web statique composé de deux fichiers :

1. `index.html` : structure HTML et styles CSS
2. `pokemon.js` : logique JavaScript et appels à la PokéAPI

L'application consomme l'API publique PokéAPI sans backend propre. Toutes les données sont récupérées en temps réel via des requêtes `fetch()` asynchrones.

Fonctionnalités

1. Recherche d'un Pokémon par numéro avec affichage de sa fiche (image officielle, types, statistiques, description en français/anglais, poids, taille)
2. Lecture du cri du Pokémon via un fichier audio `.ogg`
3. Liste de tous les Pokémon filtrée par génération (Gen 1 à Gen 9) avec navigation vers la fiche de chaque Pokémon

4. Liste des 64 baies avec leurs caractéristiques détaillées (taille, fermeté, saveur dominante, temps de croissance, douceur)
5. Comparateur de deux Pokémons côte à côte avec mise en évidence des statistiques gagnantes

Endpoints de la PokéAPI utilisés

Méthode	URL	Description
GET	/pokemon/{id}	Données principales du Pokémon
GET	/pokemon-species/{id}	Description textuelle et espèce
GET	/pokemon?limit=N&offset=N	Liste paginée des Pokémons
GET	/berry?limit=64	Liste des 64 baies
GET	/berry/{url}	Détails d'une baie spécifique

TABLE 1 – Endpoints PokéAPI consommés par le client web

Exercice 2 : Guerre des Animaux

Architecture

L'application est composée de deux parties distinctes :

1. **Backend** : API REST développée avec Django 6 et Django REST Framework, base de données SQLite
2. **Frontend** : client web statique en HTML/CSS/JavaScript (3 fichiers séparés)

La structure du projet est la suivante :

```

guerre_animaux/
  guerre_animaux/
    settings.py      # Configuration Django (CORS, auth token, DRF)
    urls.py          # Routage principal
    wsgi.py          # Point d'entree WSGI
  api/

```

```
models.py          # Modeles Army, Animal, Weapon
serializers.py     # Serialisation JSON
views.py           # Logique metier des endpoints
urls.py            # Routage de l'API
client/
  index.html       # Structure HTML et CSS
  api.js           # Appels API et authentication
  app.js           # Affichage et interactions UI
scripts/
  populate_db.py   # Peuplement BDD via scraping actani.fr
manage.py
requirements.txt
```

Modèles de données

Trois modèles constituent le cœur de l'application :

1. **Army** : armée liée à un utilisateur via une relation `OneToOneField`, avec un nom et une description.
2. **Animal** : appartient à une armée (`ForeignKey`), peut porter une arme (`ForeignKey` vers `Weapon`, nullable).
3. **Weapon** : arme indépendante avec un nom, des points de dégâts et une description.

Scraping : Données issues d'ACT'ani

Les données initiales sont scrappées depuis la page « *Qui suis-je ?* » du site <https://actani.fr>, site d'ACT'ani, éducatrice canine et instructrice Mantrailing dans le Cantal. Le script `scripts/populate_db.py` extrait les 7 animaux présentés sur le site et les répartit entre les deux armées :

Authentification

L'authentification repose sur les tokens Django REST Framework (`TokenAuthentication`). À la connexion, un token est généré côté serveur et renvoyé au client, qui le stocke dans le `localStorage` du navigateur. Ce token est ensuite envoyé dans le header `Authorization: Token <token>` de chaque requête protégée. La déconnexion supprime le token côté serveur et côté client.

Animal	Espèce	Statut	Arme assignée
Shana	Chien	Décédée	Griffe acérée
Chopin	Chien	Décédé	Bâton magique
Toundra	Chien	En vie	Hurllement perçant
Echo	Chien (48 kg)	En vie	Poids écrasant
Tostaky	Chat	En vie	Regard de chat
Rousseau	Chat	Décédé	Griffe acérée
Kovou	Chat	En vie	Regard de chat

TABLE 2 – Animaux scrappés depuis actani.fr et intégrés dans la BDD

Identifiants des comptes de test :

dresseur1 mot de passe : motdepasse123 Armée : *La Meute d'ACT'ani* (chiens)

dresseur2 mot de passe : motdepasse123 Armée : *Les Félines d'ACT'ani* (chats)

Liste des endpoints**Authentification**

Méthode	URL	Description
POST	/api/auth/login/	Connexion — retourne un token (public)
POST	/api/auth/logout/	Déconnexion — supprime le token (authenti- fié)

Gestion de l'armée (authentié)

Méthode	URL	Description
GET	/api/my-army/	Voir son armée (nom, description, nb ani- maux)
POST	/api/my-army/	Créer son armée
PUT	/api/my-army/	Modifier le nom ou la description

Gestion des animaux (authentié)

Méthode	URL	Description
GET	/api/my-army/animals/	Lister ses animaux
POST	/api/my-army/animals/	Ajouter un animal à l'armée
DELETE	/api/my-army/animals/{id}	Supprimer un animal

Gestion des armes (propriétaire de l'animal uniquement)

Méthode	URL	Description
POST	/api/animals/{id}/weapon	Assigner une arme existante ou en créer une
DELETE	/api/animals/{id}/weapon	Retirer l'arme d'un animal

Vues publiques (sans authentification)

Méthode	URL	Description
GET	/api/armies/	Toutes les armées avec animaux et armes
GET	/api/animals/	Tous les animaux
GET	/api/weapons/	Toutes les armes
POST	/api/weapons/	Créer une arme (authentifié requis)

Guide d'utilisation

Exercice 1 Client Pokémon

Aucune installation requise. Ouvrir le fichier `index.html` directement dans un navigateur web. L'application se connecte automatiquement à la PokéAPI publique.

Exercice 2 Guerre des Animaux

Prérequis : Python 3.x installé. Exécuter les commandes suivantes depuis le dossier `guerre_animaux/guerre_animaux/` :

```
# 1. Installer les dependances
pip install -r requirements.txt
```

```
# 2. Creer les migrations de l'application
python manage.py makemigrations api

# 3. Appliquer les migrations (creation de la BDD)
python manage.py migrate

# 4. Peupler la base avec les donnees scraapees depuis actani.fr
python scripts/populate_db.py

# 5. Lancer le serveur Django
python manage.py runserver

# 6. Ouvrir le client web dans le navigateur
#   Fichier : client/index.html
```

Le serveur écoute sur `http://127.0.0.1:8000`. Se connecter avec **dresseur1** ou **dresseur2** (mot de passe : `password123`).

Conclusion

Ces deux exercices nous ont permis de mettre en pratique plusieurs compétences complémentaires : la consommation d'une API publique en JavaScript côté client (Exercice 1), et la conception complète d'une API REST avec Django REST Framework accompagnée de son client web (Exercice 2). L'intégration du scraping du site ACT'ani illustre la synergie entre la collecte automatisée de données et le développement d'applications web, compétences au cœur du parcours VCOD de la SAE 4-02.